18 · K8s 资源诊断与 TiKV OOM 修复报告

项	内容
文档编号	DT-PdM-18
对象	10.167.2.175 / 10.167.2.176（K8s v1.28.2）
现象	TiKV 重启 88 / 34 次，Exit Code 137；PD 重启 2 – 3 次；TiFlash 重启 3 次
结论	不是单一 OOM，而是"资源超卖 + 驱逐 + 优雅退出窗口不足 + RWO 卷冲突"的叠加
状态	已修复并验证，回滚包在 `/root/pdm-backup`

---

1. 现象与初始假设

观察	初始假设
tikv-2 重启 88 次，Exit 137	容器内存超限被 cgroup OOM
日志显示 "Waiting for leader eviction"	退出时在腾 leader，被 SIGKILL
PD 也重启 2 – 3 次	可能 PD 自身不稳，导致 TiKV 连不上

初始假设只对了一半。 必须逐层取证。

---

2. 四层取证

2.1 第一层：节点资源与超卖

节点	CPU 分配	内存分配	实际用量
k8s-master（16C/126Gi）	13.9C（87%）	68.2Gi 请求 / 258Gi 上限（203%）	1.6C（10%）/ 98Gi（77%）
k8s-worker1（64C/126Gi）	57.3C（89%）	98.2Gi 请求 / 197Gi 上限（157%）	1.8C（2%）/ 76Gi（60%）

关键发现：内存 limit 总和是物理内存的 1.5 – 2 倍。 这意味着"每个 Pod 都以为自己能用到 limit"，一旦同时冲高，必然触发节点级 OOM。

2.2 第二层：组件资源声明

组件	request	limit	问题
TiKV ×3	1Gi	8Gi	请求过低 → Burstable → 驱逐排序靠前
TiDB	1Gi	8Gi	同上
PD ×3	256Mi	512Mi	上限过小，region 多时必 OOM
TiFlash	无	无	完全不受约束，可吃满节点

2.3 第三层：崩溃现场日志

```
[INFO] [server.rs:1742] ["Waiting for leader eviction"] [elapsed=10.02s] [leaders_count=36]
```

```
[WARN] [util.rs:721] ["connect failed"] ... pd-0 / pd-1 / pd-2 ■■ DEADLINE_EXCEEDED
[ERROR] [pd.rs:1823] ["store heartbeat failed"]
```

解读：TiKV 收到 SIGTERM 后开始腾 36 个 leader，但 PD
此时不可达，等待超时；`terminationGracePeriodSeconds` 到点后 kubelet 发 SIGKILL → Exit 137。

所以 137 不是"内存超限被杀"，而是优雅退出没退完被杀。这就是为什么 `reason` 是 `Error` 而不是 `OOMKilled`。

2.4 第四层：内核 OOM 记录

```
oom-kill:constraint=CONSTRAINT_MEMCG ... oom_memcg=.../kubepods-burstable-pod...
Memory cgroup out of memory: Killed process 68515 (k6) anon-rss:3061032kB
```

k6 是压测工具（load-testing 命名空间），被反复 OOM 杀死。load-testing 命名空间没有任何 ResourceQuota，压测可以无限吃内存 → 节点内存压力 → kubelet 驱逐 Burstable Pod → TiKV 中招。

2.5 第五层（修复过程中发现）：RWO 卷冲突

改 TiFlash 资源时，RollingUpdate 先起新 Pod、旧 Pod 仍在跑，两者挂同一个 `ReadWriteOnce` 卷 `tiflash-data` → 新 Pod 抢不到数据目录锁 → CrashLoopBackOff（Exit 1）。

这不是 OOM，是我在修复过程中引入的次生问题，如实记录。

---

3. 根因树

```
TiKV ■■■■
■■ ① ■■■■■■■■limit ■■ 157%–203%■
■   ■■ ② load-testing ■■■■k6 ■■ OOM ■■■■
■       ■■ ③ kubelet ■■ Burstable Pod
■           ■■ TiKV request ■ 1Gi■■■■■■■■→ ■■
■           ■■ PD ■■■ 512Mi → PD ■■
■               ■■ ④ TiKV ■■■ PD ■■■■■ leader ■■
■                   ■■ ⑤ gracePeriod=30s ■■ → SIGKILL → Exit 137
■■ ⑥ ■■■TiFlash RWO ■■■■■■■■■■
```

五个因、一条链。只调内存上限治不了，只加 gracePeriod 也治不了。

---

4. 修复动作（已执行）

#	动作	前	后	目的
1	TiKV request	1Gi / 0.5C	2Gi / 1C	降低被驱逐优先级
2	TiKV grace	30s	300s	留足腾 leader 时间
3	PD limit	512Mi	2Gi（request 512Mi）	防 PD 先崩
4	TiDB request	1Gi	2Gi	同上
5	TiFlash 资源	无限制	req 4Gi/2C，limit 16Gi/8C	约束最大内存黑洞
6	load-testing 配额	无	req 16Gi / limit 24Gi / 40 Pod	压测不得打爆节点
7	PriorityClass	无	pdm-data-critical=1000000	数据层最后被驱逐
8	TiFlash 更新策略	RollingUpdate	Recreate	避免 RWO 卷双挂
9	配置备份	—	`/root/pdm-backup/*.yaml`	可一键回滚

4.1 执行脚本（可复现）

```
# 1) ■■■■
kubectl create priorityclass pdm-data-critical --value=1000000 \
  --description="PdM data layer: evicted last" --dry-run=client -o yaml | kubectl apply -f -

# 2) ■■■■
kubectl -n load-testing apply -f load-testing-quota.yaml

# 3) ■■■■■■■■■■
kubectl -n infra set resources deploy/pd-0 --requests=cpu=200m,memory=512Mi --limits=cpu=1,memory=2Gi
kubectl -n infra patch deploy/pd-0 -p '{"spec":{"template":{"spec":{"priorityClassName":"pdm-data-critical","terminationGrace

kubectl -n infra set resources deploy/tikv-0 --requests=cpu=1,memory=2Gi --limits=cpu=6,memory=8Gi
kubectl -n infra patch deploy/tikv-0 -p '{"spec":{"template":{"spec":{"priorityClassName":"pdm-data-critical","terminationGra

kubectl -n infra set resources deploy/tiflash --requests=cpu=2,memory=4Gi --limits=cpu=8,memory=16Gi
kubectl -n infra patch deploy/tiflash -p '{"spec":{"strategy":{"type":"Recreate","rollingUpdate":null}}}'
```

---

5. 验证结果

检查	结果
8 个 Pod 状态	全部 1/1 Running（pd×3、tikv×3、tidb、tiflash）
TiDB 健康	`/status` 返回 `v8.5.8`，`init_stats_percentage: 100`
节点内存	master 76%，worker1 60%（修复前 77% / 60%，未恶化）
资源声明	TiKV 2Gi/8Gi、PD 512Mi/2Gi、TiDB 2Gi/8Gi、TiFlash 4Gi/16Gi
优先级	4 类组件均为 `pdm-data-critical`
重启计数	修复后未再增长（滚动更新导致的 1 – 2 次属预期）

---

6. 遗留问题（需平台方决策，不在本次擅自变更范围）

#	问题	风险	建议
R1	TiKV 是 Deployment 而非 StatefulSet	无稳定身份，Pod 重建后 store 记录残留	迁移到 StatefulSet + headless Service
R2	TiKV/TiFlash PV 为 `local-path` 10Gi，不可扩容	数据增长后写满	换 CSI 块存储或 NFS，重建 PVC
R3	kubelet 未设 `system-reserved` / `eviction-hard`	节点内存仍可能被其他负载打满	平台方统一配置
R4	集群承载教学业务（opened x/jupyterhub/dataagent-course）	与生产负载混部	PdM 独立命名空间 + 配额（已规划）
R5	TiKV `block-cache.capacity=1GB` 偏小	读放大，扫描慢	视负载提到 2 – 4GB（当前 limit 8Gi 有余量）
R6	无 TiDB 专项告警	下次 OOM 仍靠人工发现	接入现有 Prometheus：`tikv_engine_size`、`process_resident_memory_bytes`、`restart_count`

7. 给 PdM 项目的直接结论

- TiDB 可以承载主库，但前提是本次修复生效且 R1/R2 有明确时间表。`docs/17 §8.1` 的"自动回退 PG"条款继续有效。
- PdM 自己不能再犯同样的错：`deploy/k8s/pdm-prod.yaml` 已设 ResourceQuota/LimitRange，且所有容器显式声明 requests/limits。
- 监控必须覆盖重启次数，否则"88 次重启"这种事故会再次静默发生。

8. 回滚

```
# ■■■■■
kubectl -n infra apply -f /root/pdm-backup/tikv-0.yaml
kubectl -n infra apply -f /root/pdm-backup/tiflash.yaml

# ■■■■■■■■■
kubectl -n load-testing delete resourcequota load-testing-quota
kubectl delete priorityclass pdm-data-critical
```

> 回滚会恢复"无约束"状态，等于把问题放回去。除紧急情况外不建议回滚。